

eda_top_50_excerices_for_your_body_v1.3

November 18, 2025

1 EDA – Gym Exercises Dataset

1.1 Objectif

Vérifier que le fichier `gym_exercises_dataset.csv` est correctement chargé et obtenir une vue d'ensemble :

- dimensions du jeu de données (nombre de lignes et de colonnes) ;
- typage des variables (champs textuels, listes de muscles, attributs techniques) ;
- premières observations sur les valeurs manquantes et la présence éventuelle de doublons ;
- cohérence générale du corpus avant analyse statistique et préparation NLP.

1.2 Interprétation attendue

À partir de cette première inspection, il s'agira de commenter :

- la taille globale du dataset (nombre de lignes / colonnes) ;
- la nature des variables (descriptifs textuels, attributs biomécaniques, catégories d'équipements) ;
- la présence éventuelle de valeurs manquantes ou de champs textuels incomplets ;
- l'existence (ou non) de doublons dans les données brutes ;
- la cohérence générale des premières lignes (valeurs lisibles, descriptions complètes, muscles cohérents).

Cette interprétation sert de point de départ pour juger de la qualité du corpus et préparer les étapes suivantes : nettoyage, normalisation textuelle et structuration pour le traitement NLP du projet TrAIn.me.

1.3 Configuration notebook

1.3.1 Importation des librairies essentielles

```
[2]: import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub
import prince

from pathlib import Path
```

```

import sys

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from scipy import stats

# === Localiser automatiquement le package 'themes' en remontant l'arborescence
↳===
root = Path.cwd() # ex : .../train.me/src/notebooks/eda/health_fitness_dataset
themes_root = None

for p in [root, *root.parents]:
    if (p / "themes").is_dir():
        themes_root = p
        break

if themes_root is None:
    raise FileNotFoundError(
        f"Impossible de trouver le dossier 'themes' en partant de {root}. "
        "Vérifie l'arborescence : il doit exister un dossier 'themes/' quelque
↳part au-dessus."
    )

if str(themes_root) not in sys.path:
    sys.path.insert(0, str(themes_root))

print(" Ajouté au PYTHONPATH :", themes_root)
print(" Contenu de", themes_root, ":", [x.name for x in themes_root.iterdir()])

from themes.theme_train_me import set_trainme_theme

```

```

c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\.conda\Lib\site-
packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
    from .autonotebook import tqdm as notebook_tqdm

Ajouté au PYTHONPATH :
c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\notebooks
Contenu de c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\notebooks :
['eda', 'model_training', 'preprocessing', 'themes', '__init__.py']

```

Configuration d'affichage et thème TrAIn.me

```

[3]: # Options d'affichage pour les tableaux
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)

# =====

```

```
# Thème TrAIIn.me (Seaborn/Matplotlib)
# =====
USE_DARK_THEME = True # bascule possible vers un thème clair si besoin

palette_trainme = set_trainme_theme(
    context="talk",
    font_scale=1.05,
    use_dark=USE_DARK_THEME
)

palette_trainme
```

```
[3]: ['#00E5FF', '#00BFFF', '#0A84FF', '#1E90FF', '#00FFFF', '#64D8FF']
```

1.3.2 Activation du thème TrAIIn.me

```
[4]: if USE_DARK_THEME:
    palette_trainme = set_trainme_theme(context="talk", font_scale=1.05,
    ↪ use_dark=True)
else:
    sns.set_style("ticks") # style clair sobre, axes renforcés
    sns.set_context("talk", font_scale=1.05)
    palette_trainme = sns.color_palette("deep") # palette par défaut sobre
    print(" Style activé : 'ticks' → sobre, axes renforcés (présentation pro)")

# Exemple de vérification :
print("Palette active :", palette_trainme.as_hex() if hasattr(palette_trainme,
    ↪ "as_hex") else palette_trainme)
```

```
Palette active : ['#00E5FF', '#00BFFF', '#0A84FF', '#1E90FF', '#00FFFF',
'#64D8FF']
```

1.4 Chargement et inspection initiale

Cette section vise à charger le fichier `gym_exercises_dataset.csv`, vérifier ses dimensions et obtenir un premier aperçu des données brutes. Elle reprend la même démarche que pour les autres EDA du projet : contrôle du chargement, visualisation des premières lignes, statistiques descriptives de base, analyse des valeurs manquantes et des doublons, puis liste complète des colonnes disponibles.

Chargement du dataset `gym_exercises_dataset.csv`

```
[5]: # Téléchargement du dataset depuis Kaggle
dataset_path = kagglehub.dataset_download("prajwaldongre/
    ↪ best-50-exercise-for-your-body")

# Recherche du fichier CSV dans le dossier téléchargé
csv_files = [f for f in os.listdir(dataset_path) if f.endswith(".csv")]
print("Fichiers trouvés :", csv_files)
```

```
# Construction du chemin complet vers le CSV
file_path = os.path.join(dataset_path, csv_files[0])

# Lecture du fichier CSV
df = pd.read_csv(file_path)

# Informations de confirmation
print("Dataset chargé avec succès !")
print(f"Dimensions : {df.shape[0]} lignes et {df.shape[1]} colonnes\n")
```

Downloading from
https://www.kaggle.com/api/v1/datasets/download/prajwaldongre/best-50-exercise-for-your-body?dataset_version_number=1...
 100%| | 1.76k/1.76k [00:00<00:00, 1.80MB/s]
 Extracting files...
 Fichiers trouvés : ['Top 50 Excerice for your body.csv']
 Dataset chargé avec succès !
 Dimensions : 50 lignes et 8 colonnes

Aperçu des premières lignes

```
[6]: df.head(10)
```

```
[6]:
```

	Name of Exercise	Sets	Reps	
Benefit	Burns Calories (per 30 min)			\
0	Push-ups	3	15	Builds upper body
strength			200	
1	Squats	4	12	Strengthens lower
body			223	
2	Lunges	3	10	Improves balance and
coordination			275	
3	Burpees	3	10	Full body
workout			355	
4	Mountain Climbers	3	20	Improves cardiovascular
fitness			240	
5	Jumping Jacks	3	30	Improves coordination and cardiovascular
health			295	
6	Bicycle Crunches	3	20	Targets abdominal
muscles			210	
7	Dips	3	12	Strengthens triceps and
chest			180	
8	Pull-ups	3	8	Builds upper body
strength			250	
9	Russian Twists	3	20	Improves core rotation

strength

190

	Target Muscle Group	Equipment Needed	Difficulty Level
0	Chest, Triceps, Shoulders	NaN	Intermediate
1	Quadriceps, Hamstrings, Glutes	NaN	Beginner
2	Quadriceps, Hamstrings, Glutes	NaN	Beginner
3	Full Body	NaN	Advanced
4	Core, Shoulders, Legs	NaN	Intermediate
5	Full Body	NaN	Beginner
6	Core	NaN	Intermediate
7	Triceps, Chest	Parallel Bars or Chair	Intermediate
8	Back, Biceps	Pull-up Bar	Advanced
9	Core, Obliques	None or Dumbbell	Intermediate

Informations sur le typage, les valeurs manquantes et les doublons

```
[7]: # Typage et complétude globale
df_info = df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Name of Exercise                     50 non-null     object
1   Sets                                50 non-null     int64
2   Reps                                50 non-null     int64
3   Benefit                              50 non-null     object
4   Burns Calories (per 30 min)          50 non-null     int64
5   Target Muscle Group                  50 non-null     object
6   Equipment Needed                     30 non-null     object
7   Difficulty Level                     50 non-null     object
dtypes: int64(3), object(5)
memory usage: 3.3+ KB
```

1.4.1 Statistiques descriptives des variables numériques

```
[8]: display(df.describe().T)
```

	count	mean	std	min	25%	50%	75%	max
Sets	50.0	3.12	0.328261	3.0	3.0	3.0	3.0	3.0
Reps	50.0	12.78	4.559314	5.0	10.0	12.0	15.0	30.0
Burns Calories (per 30 min)	50.0	229.96	58.896991	130.0	180.0	221.5	278.0	355.0

1.4.2 Analyse des valeurs manquantes et doublons

```
[9]: # Comptage des valeurs manquantes par colonne
missing_counts = df.isna().sum().sort_values(ascending=False)

print("\n Valeurs manquantes par colonne (top 20) :")
display(missing_counts.head(20))

# Comptage des doublons ligne à ligne
n_duplicates = df.duplicated().sum()
print(f"\n Nombre de doublons (lignes strictement identiques) : \n
↳ {n_duplicates}")
```

Valeurs manquantes par colonne (top 20) :

Equipment Needed	20
Name of Exercise	0
Sets	0
Reps	0
Benefit	0
Burns Calories (per 30 min)	0
Target Muscle Group	0
Difficulty Level	0

dtype: int64

Nombre de doublons (lignes strictement identiques) : 0

Liste complète des colonnes disponibles

```
[10]: print("\n--- Liste des colonnes du dataset ---")
print(df.columns.tolist())
```

--- Liste des colonnes du dataset ---

['Name of Exercise', 'Sets', 'Reps', 'Benefit', 'Burns Calories (per 30 min)',
'Target Muscle Group', 'Equipment Needed', 'Difficulty Level']

1.5 Statistiques descriptives et distribution des variables

Cette section analyse la distribution des variables numériques du dataset, identifie les ordres de grandeur, met en évidence les valeurs extrêmes éventuelles et prépare les étapes suivantes de détection d'outliers et de corrélation.

Objectif

- Résumer la distribution des variables numériques (moyenne, médiane, étendue).
- Observer la dispersion et la variabilité des mesures.
- Identifier d'éventuelles valeurs extrêmes ou atypiques.
- Préparer la détection d'outliers (boxplots, IQR).

Interprétation attendue À l'issue de cette section, l'analyse devra mettre en évidence :

- les valeurs centrales des variables clés (nombre de séries, nombre de répétitions, calories brûlées) pour caractériser l'intensité moyenne des exercices ;
- la dispersion et la variabilité des mesures (écarts-types, min/max, quartiles), permettant d'identifier des exercices plus « légers » ou nettement plus exigeants ;
- la présence éventuelle de valeurs extrêmes ou atypiques (par exemple nombre de répétitions très élevé ou dépense calorique particulièrement importante) qui pourront faire l'objet d'un traitement spécifique ;
- les premiers ordres de grandeur utiles pour le projet TrAIn.me (plages typiques de volume d'entraînement et de dépense énergétique), en vue de préparer la recommandation d'exercices adaptés au profil de l'utilisateur.

Statistiques descriptives globales

```
[12]: # --- Colonnes numériques du dataset ---
num_cols = df.select_dtypes(include="number").columns.tolist()
print(f"Variables numériques détectées ({len(num_cols)}) : {num_cols}")

# Statistiques descriptives de base
stats_num = df[num_cols].describe().T
display(stats_num)

# Quelques repères métier
if "Sets" in df.columns:
    print(f"\nNombre moyen de séries : {df['Sets'].mean():.2f}")

if "Reps" in df.columns:
    print(f"Nombre moyen de répétitions : {df['Reps'].mean():.2f}")

if "Burns Calories (per 30 min)" in df.columns:
    print(f"Calories moyennes brûlées en 30 min : {df['Burns Calories (per 30_
↳min)'].mean():.0f} kcal")
```

Variables numériques détectées (3) : ['Sets', 'Reps', 'Burns Calories (per 30 min)']

	count	mean	std	min	25%	50%	75%	max
Sets	50.0	3.12	0.328261	3.0	3.0	3.0	3.0	4.0
Reps	50.0	12.78	4.559314	5.0	10.0	12.0	15.0	30.0
Burns Calories (per 30 min)	50.0	229.96	58.896991	130.0	180.0	221.5	278.0	355.0

Nombre moyen de séries : 3.12

Nombre moyen de répétitions : 12.78

Calories moyennes brûlées en 30 min : 230 kcal

Distribution des variables clés

```
[13]: # Sélection des variables numériques réellement pertinentes
num_vars = num_cols # toutes les variables numériques sont métier ici
print(f"Variables représentées : {num_vars}")

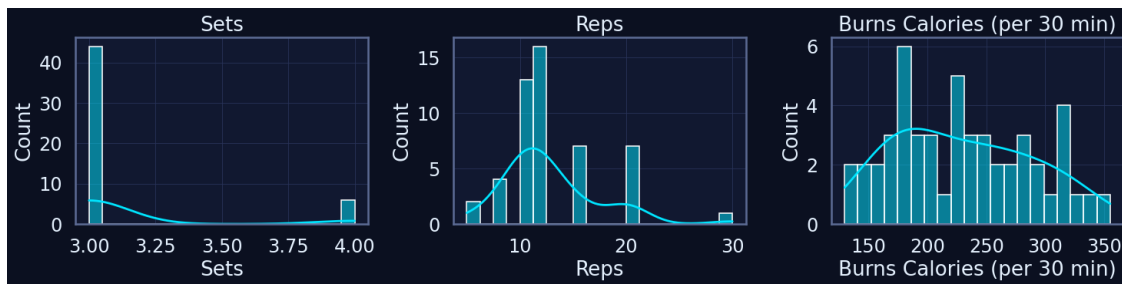
n = len(num_vars)
n_cols = 3
n_rows = int(np.ceil(n / n_cols))

plt.figure(figsize=(5 * n_cols, 4 * n_rows))

for i, col in enumerate(num_vars, 1):
    plt.subplot(n_rows, n_cols, i)
    sns.histplot(df[col], kde=True, bins=20)
    plt.title(col)
    plt.grid(True)

plt.tight_layout()
plt.show()
```

Variables représentées : ['Sets', 'Reps', 'Burns Calories (per 30 min)']

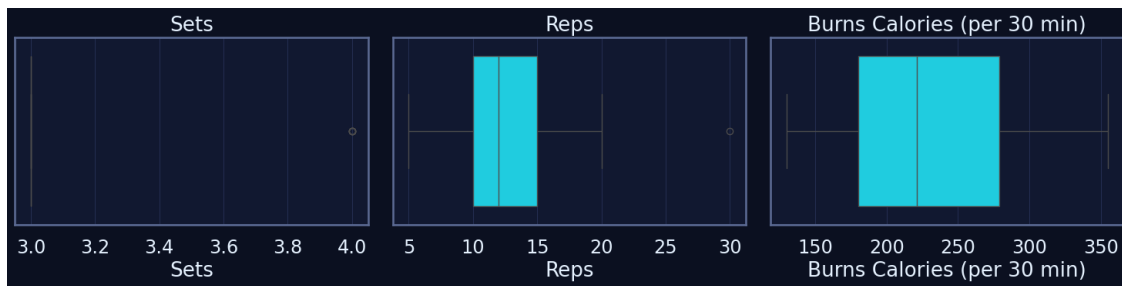


Boxplots pour repérer les outliers

```
[14]: plt.figure(figsize=(5 * len(num_vars), 4))

for i, col in enumerate(num_vars, 1):
    plt.subplot(1, len(num_vars), i)
    sns.boxplot(x=df[col])
    plt.title(col)
    plt.grid(True, axis="x")

plt.tight_layout()
plt.show()
```

Statistiques descriptives enrichies

```
[15]: if num_cols:
    desc = df[num_cols].describe().T
    desc["missing_%"] = df[num_cols].isna().mean() * 100
    desc["zeros_%"] = (df[num_cols] == 0).mean() * 100
    desc["skew"] = df[num_cols].skew()
    desc["kurt"] = df[num_cols].kurt()

    display(desc.sort_values("std", ascending=False))
else:
    print(" Aucune variable numérique disponible pour des statistiques_
    ↪enrichies.")
```

	count	mean	std	min	25%	50%	75%	max	missing_%	zeros_% \
Burns Calories (per 30 min)	50.0	229.96	58.896991	130.0	180.0	221.5	278.0	355.0	0.0	0.0
Reps	50.0	12.78	4.559314	5.0	10.0	12.0	15.0	30.0	0.0	0.0
Sets	50.0	3.12	0.328261	3.0	3.0	3.0	3.0	4.0	0.0	0.0

	skew	kurt
Burns Calories (per 30 min)	0.274614	-0.949834
Reps	1.379863	3.070059
Sets	2.411701	3.973747

Détection rapide d'outliers (IQR)

```
[16]: iqr_report = {}

for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    outliers = df[(df[col] < Q1 - 1.5 * IQR) | (df[col] > Q3 + 1.5 * IQR)]
```

```

iqr_report[col] = len(outliers)

iqr_df = pd.DataFrame.from_dict(iqr_report, orient="index",
    ↪columns=["outliers"])

iqr_df.sort_values("outliers", ascending=False)

```

```

[16]:
           outliers
Sets                6
Reps                1
Burns Calories (per 30 min)  0

```

Aperçu corrélations (sous-ensemble lisible)

```

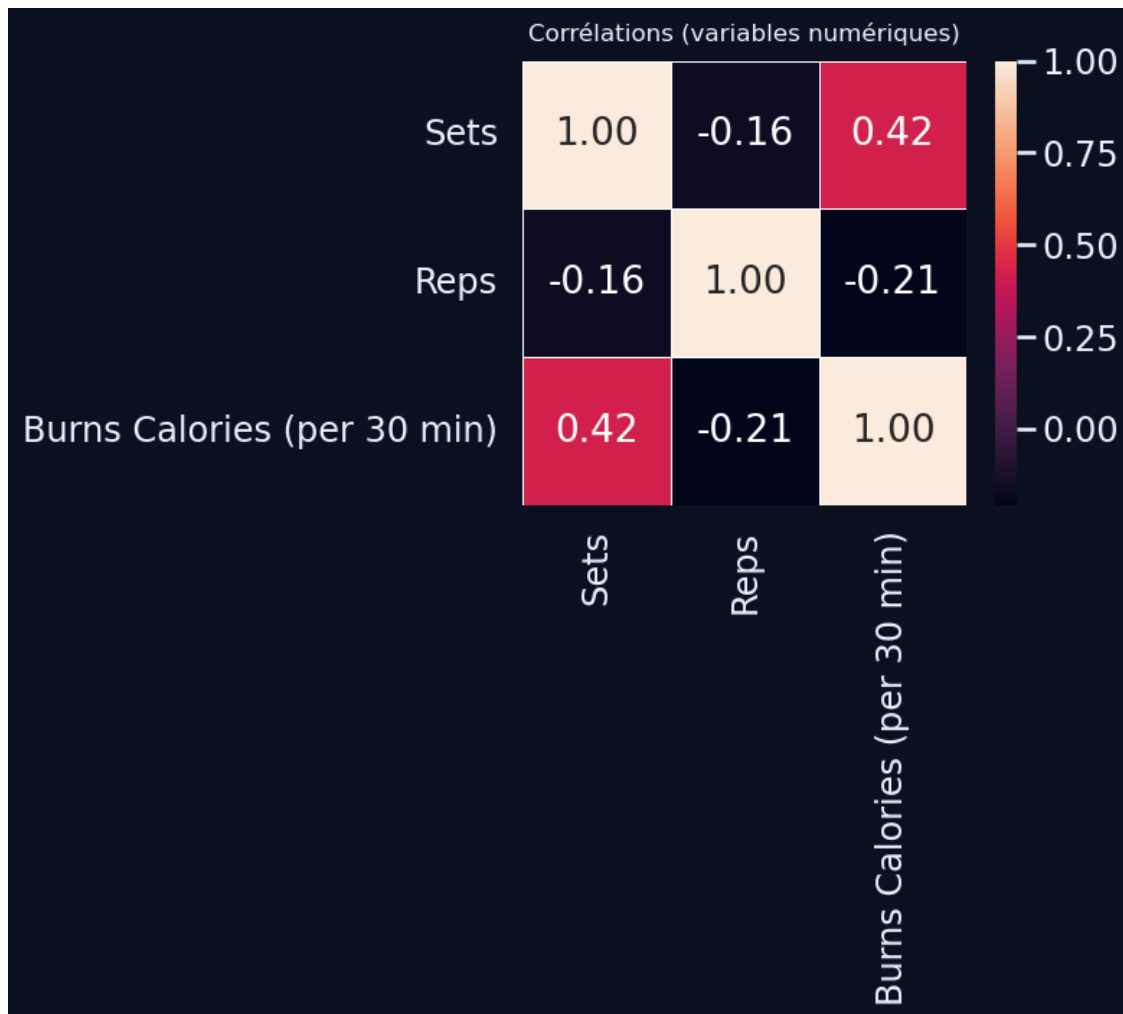
[17]: if len(num_cols) >= 2:
        corr = df[num_cols].corr(numeric_only=True)

        plt.figure(figsize=(5, 4))
        sns.heatmap(
            corr,
            annot=True,
            fmt=".2f",
            square=True,
            cbar=True,
            linewidths=0.5,
        )
        plt.title("Corrélations (variables numériques)", fontsize=12, pad=10)
        plt.tight_layout()
        plt.show()
    else:
        print(" Corrélation non pertinente : une seule variable numérique
    ↪disponible.")

```

C:\Users\fbac\AppData\Local\Temp\ipykernel_40392\4255799464.py:14: UserWarning: Tight layout not applied. The bottom and top margins cannot be made large enough to accommodate all Axes decorations.

```
plt.tight_layout()
```



1.6 Visualisation univariée

Cette section complète les statistiques descriptives par une analyse visuelle des distributions des variables numériques. L'objectif est de vérifier la forme des distributions (symétrie, concentration, valeurs extrêmes) pour les principales mesures d'intensité des exercices.

Objectif

- Visualiser la répartition des variables numériques (Sets, Reps, Burns Calories).
- Identifier les niveaux les plus fréquents et les éventuels déséquilibres.
- Repérer visuellement des valeurs extrêmes ou atypiques.
- Préparer les décisions de nettoyage ou de regroupement pour la suite de l'analyse.

Interprétation attendue À l'issue de cette section, l'analyse devra :

- décrire la forme des distributions (exercices plutôt modérés ou fortement concentrés autour de certaines valeurs de séries, de répétitions ou de calories brûlées) ;

- signaler la présence éventuelle de niveaux très peu représentés ou de valeurs extrêmes ;
- confirmer les ordres de grandeur observés (volume d'entraînement et dépense énergétique typiques) ;
- fournir des éléments visuels pour classer les exercices selon leur intensité globale, en préparation des futures étapes de recommandation.

Sélection robuste des variables numériques à visualiser

```
[19]: # --- Sélection des variables numériques pertinentes pour la visualisation
      ↪ univariée ---

num_cols = df.select_dtypes(include=["number"]).columns.tolist()
print("Variables numériques détectées :", num_cols)

# Dans ce dataset, toutes les variables numériques sont métier (volume &
      ↪ intensité)
vars_univariate = num_cols.copy()

print("Variables retenues pour la visualisation univariée :", vars_univariate)
```

Variables numériques détectées : ['Sets', 'Reps', 'Burns Calories (per 30 min)']
 Variables retenues pour la visualisation univariée : ['Sets', 'Reps', 'Burns Calories (per 30 min)']

Visualisation des distributions univariées (hist + KDE)

```
[20]: if not vars_univariate:
      print(" Aucune variable numérique pertinente à visualiser.")
      else:
          n = len(vars_univariate)
          n_cols = min(3, n)
          n_rows = int(np.ceil(n / n_cols))

          plt.figure(figsize=(5 * n_cols, 4 * n_rows))

          for i, col in enumerate(vars_univariate, start=1):
              plt.subplot(n_rows, n_cols, i)
              sns.histplot(df[col], bins=10, kde=True)
              plt.title(col)
              plt.grid(True, axis="y")

          plt.tight_layout()
          plt.show()
```

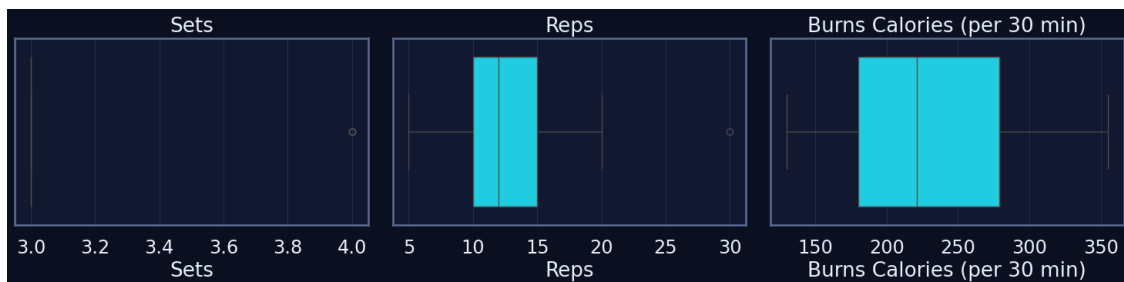


Boxplots univariés pour détection visuelle d'outliers

```
[21]: if not vars_univariate:
    print(" Aucun boxplot pertinent à afficher (pas de variable numérique_
    ↪métier).")
else:
    plt.figure(figsize=(5 * len(vars_univariate), 4))

    for i, col in enumerate(vars_univariate, start=1):
        plt.subplot(1, len(vars_univariate), i)
        sns.boxplot(x=df[col])
        plt.title(col)
        plt.grid(True, axis="x")

    plt.tight_layout()
    plt.show()
```



1.7 Visualisation bivariable

Cette section explore les relations entre variables numériques et catégorielles, afin d'identifier des tendances d'intensité selon les groupes musculaires, le niveau de difficulté ou la présence d'équipement.

Objectif

- Examiner comment les variables numériques varient selon les catégories (difficulty level, equipment, groupes musculaires).

- Détecter d'éventuels patterns (ex. exercices avancés brûlant plus de calories).
- Repérer les co-occurrences entre catégories pour mieux structurer le référentiel.
- Préparer la construction de règles métier pour TrAIIn.me.

Interprétation attendue L'analyse devra permettre :

- d'identifier les catégories associées à des intensités plus fortes (reps, calories) ;
- de vérifier si le niveau de difficulté suit une progression logique ;
- de faire ressortir des regroupements naturels entre groupes musculaires, difficulté et volume d'effort ;
- de repérer les catégories rares ou atypiques (par exemple exercices avancés mais peu gourmands en calories).

Relations catégorielles numériques (Difficulty Level)

```
[22]: # Variables numériques
num_cols = df.select_dtypes(include="number").columns.tolist()

plt.figure(figsize=(6 * len(num_cols), 5))
for i, col in enumerate(num_cols, start=1):
    plt.subplot(1, len(num_cols), i)
    sns.boxplot(data=df, x="Difficulty Level", y=col)
    plt.title(f"{col} selon Difficulty Level")
    plt.xticks(rotation=45)
    plt.grid(True, axis="y")

plt.tight_layout()
plt.show()
```



Co-occurrences entre catégories (crosstab + heatmap) Cette analyse met en évidence les combinaisons les plus fréquentes entre « Target Muscle Group » et « Difficulty Level » pour repérer les zones d'effort typiques (ex. exercices avancés orientés full-body).

```
[24]: # =====
# Préparation des catégories (Top 50 Exercises for Your Body)
# =====
```

```

df_cat = df.copy()
# On remplace les NaN d'équipement par "None" pour garder l'info "sans matériel"
if "Equipment Needed" in df_cat.columns:
    df_cat["Equipment Needed"] = df_cat["Equipment Needed"].fillna("None")

def plot_crosstab_heatmaps(
    df,
    pairs,
    top_k=12,
    normalize=None,      # None / "row" / "col" (pourcentages)
    annotate=False,      # True pour afficher les valeurs
    log_color=False,     # True pour mieux voir les faibles valeurs
    figsize=(16, 9),
    title_fontsize=13,
    tick_fontsize=9,
    label_rotation_x=30,
    label_rotation_y=0,
    cbar_shrink=0.85,
):
    """Heatmaps propres et lisibles pour analyser les co-occurrences de
    ↪catégories."""
    if not pairs:
        print(" Aucune paire de catégories disponible.")
        return

    n = len(pairs)
    ncols = 2
    nrows = (n + ncols - 1) // ncols
    fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=figsize)
    axes = np.atleast_1d(axes).ravel()

    for i, (a, b) in enumerate(pairs):
        ax = axes[i]

        # Top-k pour réduire la cardinalité (plus lisible)
        a_top = df[a].value_counts().index[:top_k]
        b_top = df[b].value_counts().index[:top_k]
        sub = df[df[a].isin(a_top) & df[b].isin(b_top)]

        ct = pd.crosstab(sub[a], sub[b])

        # Normalisation optionnelle en pourcentages
        if normalize in {"row", "col"}:
            axis = 1 if normalize == "row" else 0
            ct = (
                ct.div(

```

```

        ct.sum(axis=axis).replace(0, np.nan),
        axis=1 - axis,
    ) * 100
).round(1)

# Palette & échelle adaptées
heatmap_kwargs = dict(
    cmap="crest",
    linewidths=0.4,
    linecolor="grey",
    cbar_kws={"shrink": cbar_shrink},
    square=False,
)
if log_color and normalize is None:
    # Log uniquement pour les comptes bruts (pas en %)
    vmin = max(ct.replace(0, np.nan).min().min(), 1e-6)
    vmax = ct.max().max()
    heatmap_kwargs["norm"] = LogNorm(vmin=vmin, vmax=vmax)

# Annotations optionnelles
if annotate:
    sns.heatmap(
        ct,
        ax=ax,
        **heatmap_kwargs,
        annot=True,
        fmt=".1f" if normalize else "g",
        annot_kws={"fontsize": max(tick_fontsize - 1, 7)},
    )
else:
    sns.heatmap(ct, ax=ax, **heatmap_kwargs)

ax.set_title(
    f"Co-occurrences {a} × {b} (top {top_k})",
    fontsize=title_fontsize,
    pad=8,
)
ax.set_xlabel(b, fontsize=10)
ax.set_ylabel(a, fontsize=10)

# Ticks propres
ax.tick_params(axis="x", labelsize=tick_fontsize,
↪rotation=label_rotation_x)
ax.tick_params(axis="y", labelsize=tick_fontsize,
↪rotation=label_rotation_y)
ax.grid(False)

```



```

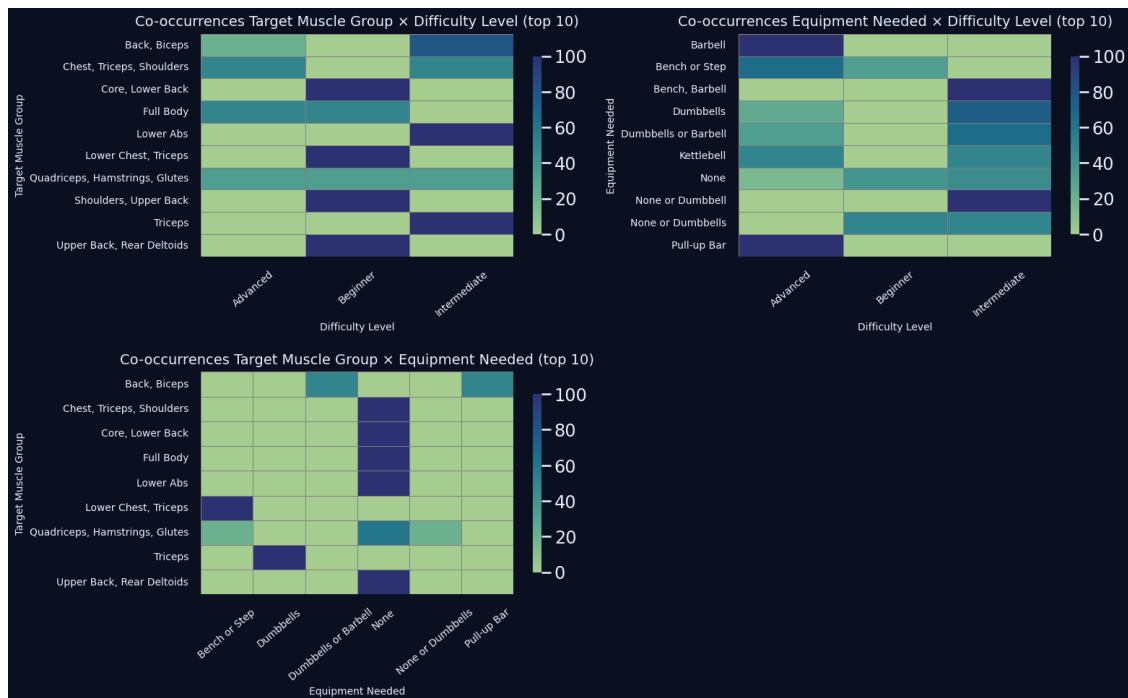
# Masquer axes vides si nombre impair
for j in range(i + 1, nrows * ncols):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()

# =====
# Sélection des paires & appel (version Top 50 Exercises)
# =====
pairs = []
if {"Target Muscle Group", "Difficulty Level"}.issubset(df_cat.columns):
    pairs.append(("Target Muscle Group", "Difficulty Level"))
if {"Equipment Needed", "Difficulty Level"}.issubset(df_cat.columns):
    pairs.append(("Equipment Needed", "Difficulty Level"))
if {"Target Muscle Group", "Equipment Needed"}.issubset(df_cat.columns):
    pairs.append(("Target Muscle Group", "Equipment Needed"))

# Paramètres recommandés pour ce dataset :
# - normalize="row" pour lire des ratios par catégorie de ligne
# - annotate=False pour éviter la surcharge visuelle
# - log_color=False car on travaille en %
plot_crosstab_heatmaps(
    df_cat,
    pairs=pairs,
    top_k=10,
    normalize="row",      # None = comptes bruts ; "row"/"col" = pourcentages
    annotate=False,
    log_color=False,
    figsize=(16, 10),
    title_fontsize=14,
    tick_fontsize=10,
    label_rotation_x=40,
    label_rotation_y=0,
    cbar_shrink=0.8,
)

```



Comptage empilé (stacked) : Equipment Needed × Difficulty Level

```
[25]: # On travaille avec une copie où les NaN d'équipement sont remplacés par "None"
df_stack = df.copy()
df_stack["Equipment Needed"] = df_stack["Equipment Needed"].fillna("None")

# On retient les équipements les plus fréquents
top equip_for_stack = df_stack["Equipment Needed"].value_counts().head(6).index
stack_subset = df_stack[df_stack["Equipment Needed"].isin(top equip_for_stack)].
↳copy()

# Crosstab : Equipment Needed × Difficulty Level
crosstab_diff = pd.crosstab(
    stack_subset["Equipment Needed"],
    stack_subset["Difficulty Level"]
)
display(crosstab_diff)

# Normalisation en pourcentage pour lecture plus intuitive
crosstab_diff_pct = crosstab_diff.div(crosstab_diff.sum(axis=1), axis=0) * 100

plt.figure(figsize=(9, 5))

bottom = np.zeros(len(crosstab_diff_pct))
x = np.arange(len(crosstab_diff_pct.index))
```

```

for diff in crosstab_diff_pct.columns:
    values = crosstab_diff_pct[diff].values
    plt.bar(
        x,
        values,
        bottom=bottom,
        label=diff,
    )
    bottom += values

plt.xticks(x, crosstab_diff_pct.index, rotation=45, ha="right")
plt.ylabel("Proportion (%)")
plt.title("Répartition des niveaux de difficulté au sein du Top Equipments")
plt.legend(title="Difficulty Level", bbox_to_anchor=(1.05, 1), loc="upper left")
plt.tight_layout()
plt.show()

```

Difficulty Level	Advanced	Beginner	Intermediate
Equipment Needed			
Bench or Step	2	1	0
Dumbbells	1	0	3
Dumbbells or Barbell	1	0	2
Kettlebell	1	0	1
None	3	8	9
None or Dumbbells	0	1	1

